Command Line Parsing

Computer Programming 1

Commands in Unix



- Much of the work on a server is done via commands typed on the terminal
- These commands accept arguments and options:
  - Arguments specify essential information for a command
  - Options customize the behavior of the command
- The possibility of passing arguments and options simplifies automation via scripts (since the need for interaction is strongly reduced)

Example



- Consider the following case, a command written in a terminal:
  `cat -n filename.txt`
- `cat` outputs a file to the terminal
- `-n` instructs the `cat` command to print line numbers
- `filename.txt` isthe file to print
- to properly work, `cat` needs to ”know” which arguments have been passed when invoking it
- These are not in `cin`, a special mechanism is required. This mechanism is via `argc` and `argv`

Command Line Arguments



- C and C++ allow to specify that a program accepts command line arguments using the `argc` and `argv` arguments

```
int main(int argc, char * argv[]) {
  ...
}
```

- `argc` (argument count) is the number of words of the command line, including the program name
- E.g., `> ./a.out 10 22.5 myfile` → `argc = 4`
- `argv` (argument vector) is an array of pointers to characters (strings) into which the command line words are copied
- The arguments are strings: if they represent numbers, they must be converted

Command Line Arguments




```
 1  #include <iostream>
 2  using namespace std;
 3
 4  int main (int argc, char * argv[]) {
 5    cout << "Argument count is: "
 6         << argc << endl;
 7    for (int i = 0; i < argc; i++) {
 8      cout << "Argument " << i
 9           << " is: " << argv[i] << endl;
10    }
11  }
```

```
./a.out -n filename.txt
Argument count is: 2
Argument 0 is: ./a.out
Argument 1 is: -n
Argument 2 is: filename.txt
```

String conversion functions



How can I convert a string into a number:

- Writing the code yourself (see next slide): simple cases are simple, then you need to take care of a lot cases (negative numbers, comma for floats, ...)
- Using a library function
  - Some functions straight from C, operating on `char *`
  - Other functions defined in C++, operating on strings and with a more complex behaviour (e.g., conversion from binary)

Command Line Arguments: from first principles



When passing strings, we don't care about passing size (unless we want to replicate the n version of some `cstring` functions, e.g., `strncat`)

```
#include <iostream>
using namespace std;

unsigned long convert(char* string) {
  unsigned long value = 0;
  for (int i = 0;
       string[i] >= '0' && string[i] <= '9';
       i++) {
    value = value * 10 + (string[i] - '0');
  }
  return value;
}

int main (int argc, char * argv[]) {
  for (int i = 1; i < argc; i++) {
    long number = convert(argv[i]);
    cout << argv[i] << " => " << number << endl;
  }
```

Notice that we can't distinguish `convert("a", 1)` from `convert("0", 1)`.

```
108 => 108
204 => 204
305 => 305
4050 => 4050
aaa => 0
a10 => 0
```

C-string conversion: `atoi`




```
int atoi(const char* str)
```

- `atoi`: convert a string to an integer
- `str` is the string (array of chars terminated by `'\0'`) to convert
- It returns `0` if the string cannot be converted
- It stops at the first not valid char
- defined in `<cstdlib>`

```
#include <stdlib>
#include <iostream>
using namespace std;

int main (int argc, char *argv[]) {
  for (int i = 0; i < argc; i++) {
    int n = atoi(argv[i]);

    cout << "argv[ " << i << "] is: "
         << argv[i] << ": " << n
         << " 2*n is: " << 2 * n << endl;

    return 0;
}
```

C-string conversion: `atof`




```
int atof(const char* str)
```

- `atof`: convert a string to an integer
- `str` is the string (array of chars terminated by `'\0'`) to convert
- It returns `0.0` if the string cannot be converted
- It stops at the first not valid char
- defined in `<cstdlib>`

```
#include <stdlib>
#include <iostream>
using namespace std;

int main (int argc, char *argv[]) {
  for (int i = 0; i < argc; i++) {
    double n = atof(argv[i]);

    cout << "argv[ " << i << "] is: "
         << argv[i] << ": " << n
         << " 2*n is: " << 2 * n << endl;

    return 0;
}
```

Wait, wait...

```
int main () {
    int n;
    char buffer[256];
    cout << "Enter a number: " << endl;
    cin >> buffer;
    n = atof(buffer);
    cout << "Value is: " << n << " and n * 2 is: " << 2 * n << endl;

    return 0;
}
```

I enter `10.10` and I get `20`. Why?

(Solution: n is declared as integer)

String conversion: `stoi`

```
int stoi(const string& str,
size_t* idx = 0, int b = 10)
```

- `stoi`: convert a string to an integer
- arguments:
  - `str` string to convert. As reference for efficiency
  - `idx` (optional) used to track how many characters were read
  - b (optional) base (default is 10)
- in case of error throws `invalid_argument` exception

```
int main () {
    string str1 = "2024, CP1";
    string str2 = "0x8A";
    string str3 = "-100";
    string str4 = "100";

    int num1 = stoi(str1); // stoi(str, nullp
    size_t idx;
    int num1c = stoi(str1, &idx, 10); // deci
    int num2 = stoi(str2, nullptr, 16); // he
    int num3 = stoi(str3, nullptr, 2); // bin
    int num4 = stoi(str4, nullptr, 2); // bin

    cout << str1 << ": " << num1 << endl; //
    cout << str1 << ": " << num1c << " " << i
    cout << str2 << ": " << num2 << endl; //
    cout << str3 << ": " << num3 << endl; //
    cout << str4 << ": " << num4 << endl; //
}
```

Handling exceptions




```
#include <string>
#include <iostream>
using namespace std;

int main () {
  // error: invalid argument
  string strInv = "CP1 2024";

  try {
    int numInv = stoi(strInv);
  }
  catch(invalid_argument) {
    cout << "Invalid argument raised for "
         << strInv << endl;
  }

  return 0;
}
```

Output:

```
Invalid argument raised for CP1 2024
```

Other Conversion functions



- `stol(const string& str, size_t* idx = 0, int b = 10)` String to long
- `stof(const string& str, size_t* idx = 0, int b = 10)` String to float
- `stod(const string& str, size_t* idx = 0, int b = 10)` String to double
- `stoul(const string& str, size_t* idx = 0, int b = 10)` String to unsigned long

Managing options, Optional part



The following is a in-depth, optional part

Command options



- Command options always start with `-` followed by a character identifying the option, or `--` and a string identifying the option. Example: `cut -n`, `cut --complement`
- Some options take one or more arguments. Example: `cut -f 2,3`
- When using short options, these can be grouped under the same `-`. Example: `cut -ns` is equivalent to `cut -n -s`

Getting help



- Nearly all commands provide a `--help` (`-h`) and a `--verbose` (`-v`) option.
- If installed, the `man` command provides a more verbose description of a command and its options.

```
cut --help
ls --help
gcc --help
man gcc
```

Piping and Redirects



- Unix provides two powerful ways to combine commands: piping and redirects.
- Piping and redirects allow you to chain commands to perform sequences of transformations on a given input.
- Piping and redirects allow you to enforce the KISS (Keep It Simple and Stupid) principle: each command in Unix does a simple thing and does it really well. If complex behavior is required, simple commands are combined.

Piping



- Piping allows you to send the output of a program as input to another: whatever a program outputs to `stdout` becomes the `stdin` of the program next in the pipe.
- Piping uses `|` to combine commands.

```
# the output of cat is the input of grep
# the output of grep goes to cut
cat /etc/passwd | grep -v nologin | cut -f1 -d:
```

Redirects



- Redirects allow you to save `stdout` to a file or take `stdin` from a file. It uses > (save output to file), >> (append output to file), and < (take input from file).

```
# save stdout of next command to tmp.txt
cat /etc/passwd > tmp.txt
# take tmp.txt as stdin for the grep command; grep outputs to tmp2.txt
grep -v nologin < tmp.txt > tmp2.txt
# take tmp2.txt as stdin for cut
cut -f1 -d: < tmp2.txt
```

Return Value



- Unix commands by convention return: `0` if no error, a negative number in case of error, a positive number in case of success.
- There are exceptions, of course (e.g., for instance a function returning a number)
- You can inspect the value returned using `$?` (bash). For instance, from the terminal:

```
ls /tmp > /dev/null
echo $?
# will return 0 (success)

ls /non-existent-folder
echo $?
# will return 2 (non existent folder)
```

Parsing options



- Parsing command options (as opposed to arguments) is usually a pain
- Options:
  - might be present or not
  - might appear in any order
  - some require arguments, other might have arguments, other do not
- Writing code from scratch is difficult (*"if current argument is an option with argument, check next, otherwise ..."*)
- There are libraries!

`getopt`



- Works with short options only.
- Defined in `unistd.h`, which you need to include in your app.
- Uses a special encoding to specify which options your program accepts: a character for each option, optionally followed by `:` if the option requires an argument. Example: `vo:` (options `v` and `o`, with `o` requiring an argument).

Short Options



- `getopt(argc, argv, opts)` reads the next option, returning -1 if done.
- `optarg` holds the argument associated with the current option (global variable).
- `optind` global variable holds the index of the first argument not associated with any option.

```
while ((opt=getopt(argc,argv,"vo:")) != -1) {
  switch(opt) {
  case 'v':
    verbose = true;
    break;
  case 'o':
    output = optarg;
    break;
  default:
    // handle error
    break;
  }
}
```

Long Options (`getopt_long`)



- Used to parse descriptive options prefixed with `--` (e.g., `--output` file.txt).
- Defined in `getopt.h` (GNU extension).
- Uses an array of `struct option` to define long options, their argument requirements, and their flag mapping.

```
#include <getopt.h>

static struct option const long_options[] = {
  { "verbose", no_argument, NULL, 'v' },
  { "output", required_argument, NULL, 'o' },
  { NULL, 0, NULL, 0 }
};
```

Using `getopt_long` in the Event Loop



- Pass the long options array alongside the short option string into `getopt_long`.
- Track the current option index using a pointer parameter (`&option_index`).

```
int opt;
int option_index = 0;

while ((opt = getopt_long(argc, argv, "vo:", long_options, &option_index)) != -1) {
  switch (opt) {
  case 'v':
    verbose = true;
    break;
  case 'o':
    output_file = optarg;
    break;
  default:
    // getopt_long automatically handles error messaging
    return 1;
  }
}
```

Processing Remaining Arguments (`optind`)



- Once option parsing completes, `optind` points to the first non-option command-line argument.
- Use `optind` to iterate through leftover positional arguments (e.g., input file paths, commands).

```
// Process remaining positional arguments after options
for (int i = optind; i < argc; i++) {
  std::cout << "Positional argument: " << argv[i] << std::endl;
}
```

GNU Extensions & Availability



- Linux: `getopt_long` is part of the GNU C Library (glibc) and is available out-of-the-box.
- macOS: macOS uses a BSD-derived libc that includes full support for $\text{getopt}_{\text{long}}$ and <getopt.h>.
- Windows (MSVC): No. Native Microsoft Visual C++ (MSVC) does not include getopt.h or $\text{getopt}_{\text{long}}$. However, it is available if you compile using MinGW, Cygwin, or if you pull in a third-party wrapper library via a package manager like vcpkg